



Artificial Intelligence Term Project Report

Student Name: Merve ADALI

Student ID: 211805049

Project Topic: Disaster Rescue Operation Simulation with an LLM-Based Multi-Agent System

1. Abstract and Project Objective

The objective of this project is to design an LLM-based Multi-Agent System (MAS) to coordinate autonomous rescue robots during disaster scenarios. The system consists of three distinct reasoning modules (Planner, Assignment, and Critic) that analyze tasks in a disaster area, generate a rescue plan, assign tasks based on the hardware capabilities of the robots, and evaluate these assignments according to predefined rules. The project successfully simulates autonomous planning, heuristic search (Informed Search), and an iterative feedback loop between AI agents.

2. System Architecture and Agent Roles

The project utilizes the Google Gemini API to create three fundamental agents that communicate with each other using the JSON data format:

- **Planner Agent:** Analyzes randomly generated tasks in the disaster area (e.g., clear debris, deliver medicine, map building) and creates an optimal execution plan based on urgency and spatial costs.
- **Assignment Agent:** Takes the ordered list from the Planner and assigns tasks to the available rescue robots based on their specific capability sets.
- **Critic Agent:** Audits the assignments made by the Assignment Agent against system rules. If a mismatch is detected, it requests a revision from the Assignment Agent using a "Revisions Required" command. Once all assignments are flawless, it outputs "DONE" to initiate the operation.

3. Priority Handling and Routing

The execution sequence of tasks in the system is determined dynamically, not only by static priority labels (high, medium, low) but also by the spatial distance of the targets.

As specified in the project requirements, the rule "*high priority tasks should be handled first*" is strictly enforced by the Planner Agent, ensuring that rescue victim tasks are always placed at the top of the list.

Informed Search Integration: When a conflict occurs between tasks of the same priority (e.g., two medium priority tasks), a heuristic search infrastructure is utilized to maximize system efficiency. The **Manhattan Distance** from the center for each task is calculated in Python using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and provided to the agents as an input parameter. In cases of identical priority, the Planner Agent places the task with the shortest distance (lowest distance value) first, thereby providing an autonomous time and speed optimization.

4. Task Completion Efficiency

During the task assignment phase, rather than a random distribution, a 100% match is targeted by considering the specific capabilities of the robots.

- map building tasks $\rightarrow$ assigned to **R1** (Search and mapping - Fast) robot.
- rescue victim and deliver medicine tasks $\rightarrow$ assigned to **R2** (Medical support - Medium) robot.
- clear debris tasks $\rightarrow$ assigned to **R3** (Heavy debris removal - Slow) robot.

This autonomous coordination between agents prevents resource waste. For instance, time-consuming tasks like heavy debris removal are strictly assigned to the slow, specialized robot, preventing the fast medical robots from being unnecessarily occupied. The Critic Agent's auditing and approval of this alignment prove the accuracy of the system's task completion efficiency.

5. Execution Phase and Cost Calculation

In accordance with the project requirement stating that "*the final rescue plan is generated and applied by the robot agents*," the planned and approved tasks are virtually executed in the final stage of the simulation.

During this phase, each robot's speed class (Fast=3, Medium=2, Slow=1) and the task's Manhattan distance are processed. The task completion cost for the robots is calculated using the following formula:

$$\text{Estimated Time} = \frac{\text{Distance}}{\text{Robot Speed}}$$

This approach converts hardware speed parameters into an operational time cost, analytically satisfying the "relevant speed" requirement of the project.

6. Simulation Results and Analysis

Below is the terminal output showing the autonomous decision-making mechanism generated by the agents and the operational execution phase resulting from running the system:

=== *WORLD MODEL AND INFORMED SEARCH INITIALIZATION* ===

Tasks (With Calculated Distances):

```
[  
  {  
    "id": "T1",  
    "type": "map building",  
    "priority": "low",  
    "location": [  
      0,  
      1  
    ],  
    "distance": 1  
  },  
  {  
    "id": "T2",  
    "type": "map building",  
    "priority": "low",  
    "location": [  
      2,  
      7  
    ],  
    "distance": 9  
  },  
  {  
    "id": "T3",  
    "type": "map building",
```

```
"priority": "low",
"location": [
  3,
  1
],
"distance": 4
},
{
  "id": "T4",
  "type": "rescue victim",
  "priority": "high",
  "location": [
    10,
    0
  ],
  "distance": 10
},
{
  "id": "T5",
  "type": "deliver medicine",
  "priority": "medium",
  "location": [
    8,
    3
  ],
  "distance": 11
}
]
```

=====

--- 1. PLANNER AGENT: CREATING STRATEGY ---

Planned Sequence:

[{"id": "T4", "type": "rescue victim", "priority": "high", "location": [10, 0], "distance": 10}, {"id": "T5", "type": "deliver medicine", "priority": "medium", "location": [8, 3], "distance": 11}, {"id": "T1", "type": "map building", "priority": "low", "location": [0, 1], "distance": 1}, {"id": "T3", "type": "map building", "priority": "low", "location": [3, 1], "distance": 4}, {"id": "T2", "type": "map building", "priority": "low", "location": [2, 7], "distance": 9}]

--- 2. ASSIGNMENT AGENT: DISTRIBUTING TASKS ---

Proposed Assignments:

{"T4": "R2", "T5": "R2", "T1": "R1", "T3": "R1", "T2": "R1"}

--- 3. CRITIC AGENT: AUDIT AND APPROVAL ---

Critic Feedback (Round 1): DONE

SYSTEM APPROVAL RECEIVED! RESCUE OPERATION INITIATED. <<<

=== 4. EXECUTION PHASE (ROBOTS ON MISSION) ===

-> Robot R2 deployed to coordinates [10, 0].

[Action]: rescue victim (Priority: high)

[Cost]: Distance 10 units, Estimated Time: 5.0 time units.

-> Robot R2 deployed to coordinates [8, 3].

[Action]: deliver medicine (Priority: medium)

[Cost]: Distance 11 units, Estimated Time: 5.5 time units.

-> Robot R1 deployed to coordinates [0, 1].

[Action]: map building (Priority: low)

[Cost]: Distance 1 units, Estimated Time: 0.3 time units.

-> Robot R1 deployed to coordinates [3, 1].

[Action]: map building (Priority: low)

[Cost]: Distance 4 units, Estimated Time: 1.3 time units.

-> Robot R1 deployed to coordinates [2, 7].

[Action]: map building (Priority: low)

[Cost]: Distance 9 units, Estimated Time: 3.0 time units.

=== ALL TASKS SUCCESSFULLY COMPLETED! ===

Output Analysis:

As seen in the simulation results above, the Planner Agent correctly sorted the tasks by priority and distance, the Assignment Agent made the capability matches according to the rules, and the robots successfully completed the operation by calculating their estimated arrival times based on their speed parameters.

7. Conclusion

This developed project proves that Large Language Models (LLMs) can do more than just generate text; when supported by proper system instructions (Prompt Engineering) and mathematical parameters (such as Manhattan Distance), they can logically manage a complex disaster operation. The system autonomously and flawlessly fulfilled all requirements specified in the assignment guidelines, including priority handling, task completion efficiency, capability matching, and practical execution.